SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This section introduces the proposed system, a Loan Eligibility Assessment application developed to address and help the users from rural areas, especially individuals who have mini stalls . The system leverages a streamlit-based user interface and a large language model (LLM) to assess business loan applications based on the users financial and operational inputs.
With this solution we aim to provide a fast, consistent and explainable loan decisions particularly for small and micro business scenarios where traditional evaluation may be slow or inconsistent

1.1 Purpose:

The purpose of this System Analysis Document is to outline the **t**echnical scope, system architecture, and design decisions behind the development of the Loan Eligibility Assessment Application.
So, the key element of system that has been covered are as followed:

1. Architecture:

The system follows a **lightweight client-server architecture**:

- A **Streamlit frontend** is used to collect structured user inputs (business and financial data)
- A **backend API (FastAPI)** processes requests and communicates with the LLM
- The LLM performs the loan eligibility evaluation based on prompting logic

This modular design ensures:

- Ease of development and deployment
- Scalability for future integration (e.g., databases or cloud hosting)
- Clear separation between UI, logic, and AI processing

2. Data Flows:

The data flow within the system is as follows:

1. User enters business and financial details via the Streamlit interface
2. Input data is validated and structured into a predefined format

© UMHackathon 2026

3. The data is sent to the backend API endpoint
4. The backend constructs a prompt and sends it to the LLM
5. The LLM generates a loan decision (approved/rejected with reasoning)
6. The response is returned to the frontend and displayed to the user

3. Model Process:

The model handles the core evaluation workflow:

- Receives structured input (e.g., sales, expenses, years of operation)
    - Applies contextual reasoning using predefined prompting strategy
    - Evaluates financial viability and risk factors
    - Generates a decision:
    a. Approval / Rejection
    b. Supporting explanation

4. Role of Reference:

This document serves as a technical reference for, developers implementing the system, testers validating system behavior, stakeholders understanding system architecture and workflow

1.2 Background:

Many small businesses, particularly informal or micro-enterprises, face challenges in obtaining loan approvals due to lack of structured financial evaluation, manual and inconsistent decision-making and limited access to financial advisory tools. This system is designed to **automate and standardize loan evaluation**, making it more accessible and efficient

Previous Version:
Currently most of the loan evaluation for small businesses is often conducted manually based on inconsistent criteria which is also very time consuming and prone to human bias. There is typically no simple digital tool available for quick preliminary loan assessment

© UMHackathon 2026

The new system introduces streamlit-based interactive user interface for structured data input with the integration of LLM for intelligent decision-making. Backend API layers have been used to support and manage communication and processing.

The new capabilities that have been introduced are automated loan eligibility assessment that provides instant feedback with clear explanation . It is also structured based on financial evaluation logic making it a scalable design for future enhancement such as integrating database and analytics

○ Target Stakeholder

Stakeholders	**Roles**	**Expectations**
User (applicant)	Inputs businesses and financial data to asses loan eligibility	Fast, simple and clear loan decision with understandable reasoning
System Admin	Monitors system usage and ensures smooth operations	Reliable system performance and accurate outputs
Development team	Builds and maintains the application	Modular codebase, clear APU structure, and maintainable architecture
QA team	Tests system functionality abd validates outputs	Defined input/output formats, proper validations, and handling of the edge cases

© UMHackathon 2026

- **System Architecture & Design**

○ **High Level Architecture**

■ **Overview:**

Type	Details
System	Desktop Website, Mobile Website
Architecture	Cloud-based microservices Architecture

The KreditX uses a cloud-based client-server architecture optimised for rural accessibility and stakeholder efficiency. It consists of a mobile website for rural entrepreneurs or B40 users and a desktop website for stakeholders. Both are powered by Streamlit frontend and connected to Python and FastAPI backend microservices for seamless data handling and AI-driven credit assessment through Gemini LLM (gemini-2.5-flash) API. Structured data is stored in a SQLite database, while files are kept in cloud storage. Deployed on a platform like AWS RDS, this architecture ensures efficient interaction between the frontend, backend, database, and AI prompt engineer.

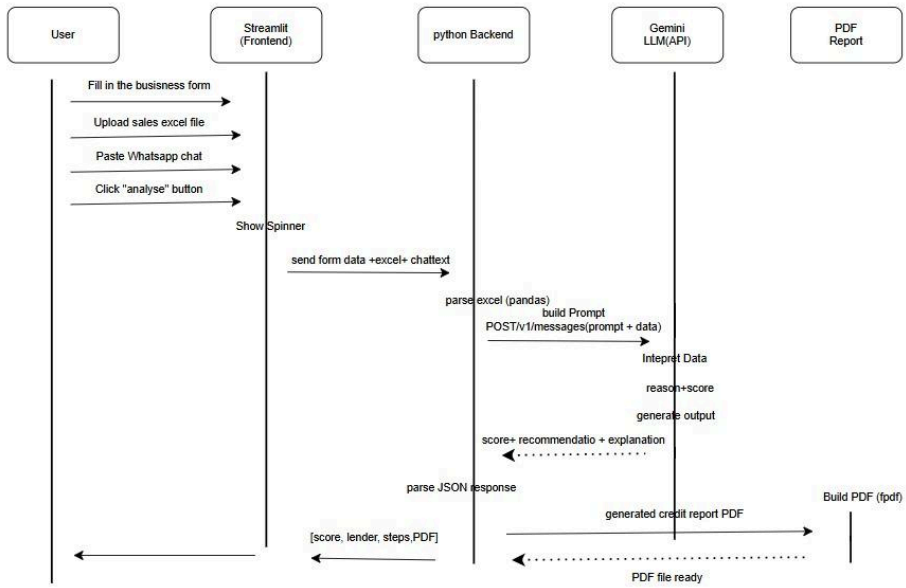
2.1.2 LLM as Service Layer

The system integrates Gemini LLM (gemini-2.5-flash) as a dedicated service layer rather than a generic AI module. The backend communicates with the LLM through API, sending structured business data for analysis. The LLM is responsible for generating human-readable insights, risk explanations, and decision support recommendations. This design ensures modularity and explainability, as the AI component is decoupled from core system logic and can be replaced or updated independently without affecting the overall architecture.

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic “AI Module”. The following Dependency diagram includes:

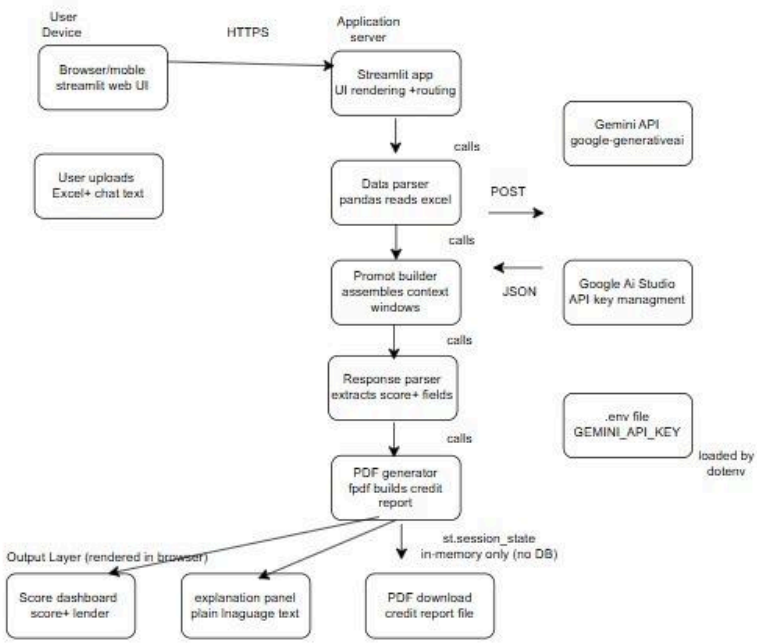
2.1.3 Dependency Diagram

© UMHackathon 2026



a.

2.2.2. Sequence Diagram



a.

- **Technological Stack**
  - **Frontend:** Streamlit- Acts as an interactive user interface for data input and output.
  - **Backend:** Python + FastAPI- Used to handle API endpoints, business login processing, and integration between AI and LLM models.
  - **Database:** SQLite- Used to store structured data in KreditX, hosted on AWS RDS for reliable persistence and querying across microservices.

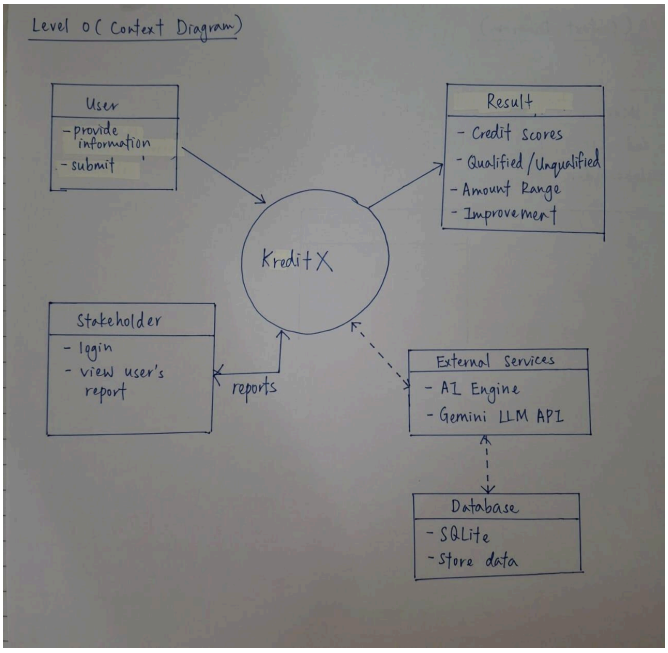
- **Development Tools:** VS Code- Used as a Python interpreter integration, debugging tools, and extension support.

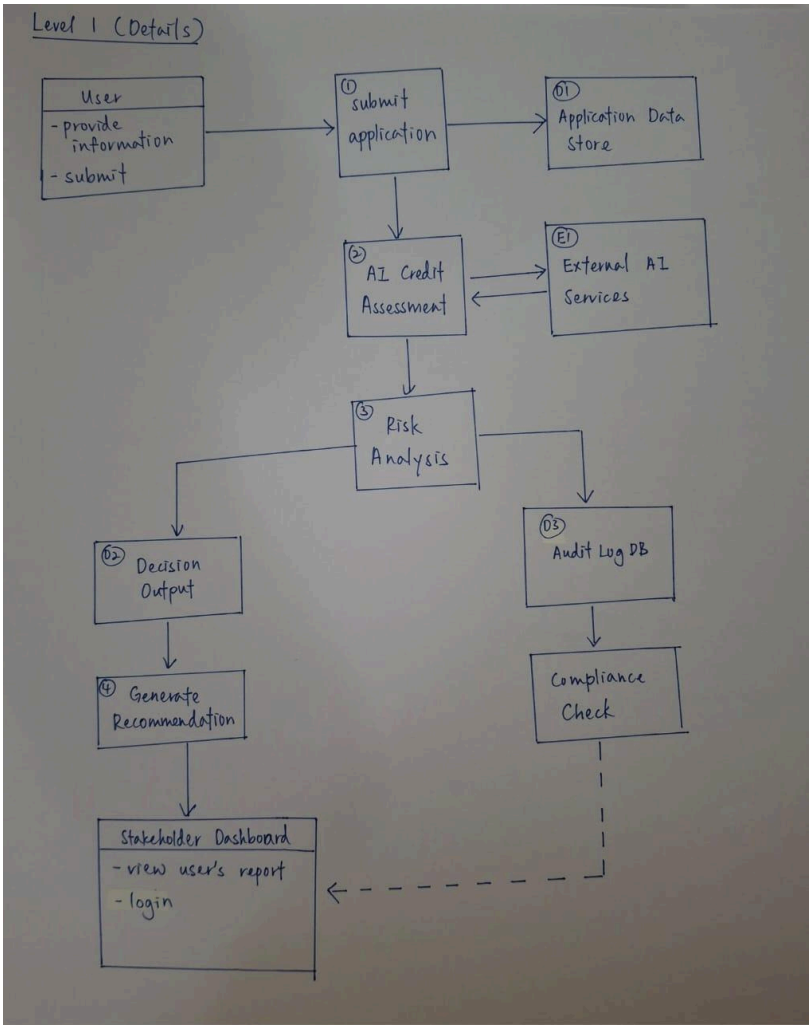
- ***Key Data Flows***
  - ***Data Flow Diagram (DFD)***

© UMHackathon 2026

Email: umhackathon@um.edu.my

a.

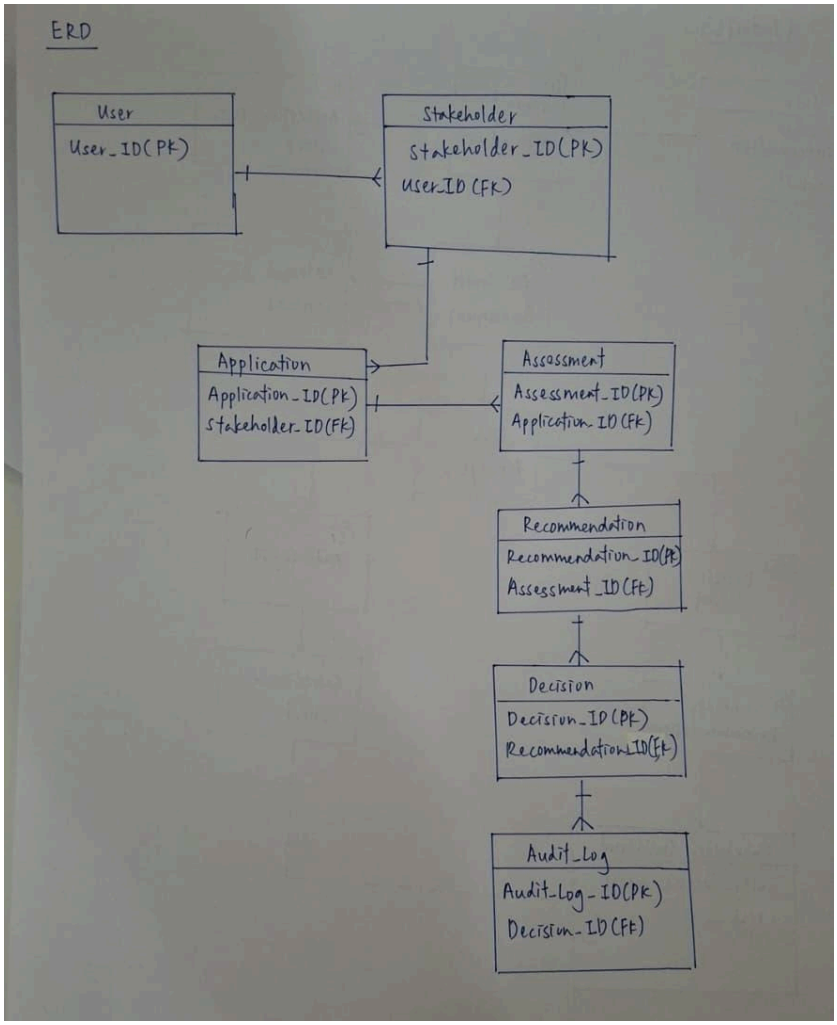




© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Normalized Database Schema**

a. 

Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

Minimum Viable Product:

- Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026. **[e.g. in Table]**

#	Features	Description
1	Data Ingestion	Accepts WhatsApp text, voice-to-text, and OCR inputs from micro-entrepreneurs.
2	AI Credit Scoring	Generates credit score based on income, expenses, and cashflow patterns.

© UMHackathon 2026

3	Loan Recommendation	Suggests loan amount, interest rate, and repayment duration based on risk profile.
4	Risk Detection	Identifies anomalies, inconsistent, or potentially fraudulent financial data.
5	Decision Output	Provides final output including credit score, approval status, and explanation.

- **Non-Functional Requirements (NFRs)**
  Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality	**Requirements**	**Implementation**
Scalability	The system must handle multiple simultaneous loan applications without performance degradation, especially during peak usage.	Cloud-based deployment with auto-scaling and load balancing.
Reliability	Credit scoring results must remain consistent and reproducible for identical input data.	Rule-based validation combined with version-controlled AI prompts.
Maintainability	System components must be modular and independently maintainable for future updates.	Microservice architecture with separated services and CI/CD pipelines.
Token Latency	AI response time must be fast enough for near real-time user experience.	Async processing, caching of repeated requests, and strict timeout handling.
Cost Efficiency	AI processing must remain within a controlled operational budget per request.	Prompt optimization, input trimming, and reuse of cached AI outputs.

- **Out of Scope / Future Enhancements**

The current MicroCredit AI system focuses only on the core credit evaluation pipeline and does not include real-time communication features. This means functionalities such as live chat between loan officers and applicants, notifications via messaging systems, or interactive support systems are not part of the current MVP. Similarly, mobile application development (Android/iOS) is excluded as the system is designed for a web-based demonstration during UMHackathon 2026.

In the long-term vision, the platform aims to integrate deeper financial ecosystem capabilities such as direct loan disbursement with banks or e-wallet providers, integration with national credit bureaus, and automated identity verification using biometric systems like facial recognition or IC scanning. Additional future enhancements may include multi-language voice assistants for rural dialects, financial literacy recommendation tools, and institutional dashboards

© UMHackathon 2026

for banks to analyze loan portfolio performance. These features are planned beyond the hackathon scope and represent the scalable direction of the system.

- **Monitor, Evaluation, Assumptions & Dependencies**

  - **Technical Evaluation:**
    - **Grayscale Rollout & A/B Testing**:

      The credit assessment and AI evaluation feature will be deployed gradually using grayscale rollout:

      - Phase 1: Release to 5% of selected users
      - Phase 2: Expand to 25% after monitoring system stability
      - Phase 3: Increase to 50% and 100% if no anomalies detected

      Monitored metrics:

      - Credit scoring error rate
      - Application completion rate
      - Average response time
      - Session persistence stability
      - AI recommendation consistency

      Two scoring logic variants are tested (A/B Testing):

      - Variant A: Rule-based credit scoring system
      - Variant B: Rule-based scoring and Gemini 2.5 Flash AI enhanced risk interpretation

      Evaluation criteria:

      - Accuracy of credit recommendations
      - Processing speed
      - False approval or rejection rate
      - User completion rate

      The better performing variant will be promoted to production.

    - **Emergency Rollback (ER) & Golden Release**:

      A Golden Release is maintained as a fallback version.

      Emergency rollback (ER) will be triggered if:

© UMHackathon 2026

- Credit scoring error rate > 3%
- System response time consistently > 5 seconds
- Session state failure affecting user flow
- Gemini API service disruption
- Application crash or deployment failure detected

The rollback strategy is designed to ensure system reliability in the event of failures. When an anomaly is detected, the system will automatically revert to the Golden Release version to restore a stable state. During this process, the AI-enhanced scoring layer powered by Gemini 2.5 Flash will be temporarily disabled, and the system will switch to a rule-based scoring fallback mode to ensure continuity of service. Once stability is confirmed, full system functionality will be restored before redeployment.

■ Priority Matrix:

Priority Matrix helps prioritize system issues based on severity and impact. This ensures that critical problems are addressed immediately while less important issues are handled in an orderly manner.

P1: If GEMINI API failure, scoring failure >3% or system crash , emergency rollback to Golden release.
P2: If response time >5s or form failure >2% , immediate fix and monitoring escalation.
P3: If minor UI bugs or translation inconsistencies, scheduled patch.
P4: If design issues, improve in future.

○ Monitoring:

System will be continuously monitored using runtime performance tracking:

A. Monitored Metrics:

- API latency (Gemini 2.5 Flash response time)
- Credit scoring success or failure rate
- User submission completion rate
- Session state persistence
- System error logs

B. Alert Conditions:

- Response time exceeds threshold
- API error rate increases
- AI inference failure detected
- Session loss or data inconsistency

C. Incident Response Flow:

1. System detects anomaly
2. Alert triggered to monitoring layer
3. Priority Matrix classification applied
4. Mitigation or rollback executed
5. Post-incident review conducted

© UMHackathon 2026

Assumptions:

- Users have stable internet connection
- Users can provide accurate business and financial data
- Uploaded documents are valid and readable
- Gemini 2.5 Flash API is available during inference
- System operates under moderate traffic load
- Stakeholders have access to dashboard devices

External Dependencies:

Tools	Purpose	Risks
Gemini 2.5 Flash API	AI reasoning, credit evaluation, and unstructured data analysis	High: API latency or failure affects scoring accuracy
Streamlit Framework	Frontend web application interface	Low: framework dependency but stable
Python Standard Libraries	Core logic support (random, datetime, string)	Low

- ***Project Management & Team Contributions***

  The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

- ***Project Timeline:*** *Show the timeline of development and justify accordingly [e.g. Day 1-2, Project Planning etc.]*

Day	Phase	Activities
1-2	Define Problem Statement	Do research on relevant topics and what problems people are struggling with Meetup with mentors
3-5	Backend Design	Connect to Gemini API
6-7	Frontend & Integration Development	Build a simple website
8	Testing	Check if everything works correctly
9	Bug Fixing	Fix the bug and make sure the fix worked

© UMHackathon 2026

10	Documentation	Finish the System Analysis, Sample Testing Analysis and Product Requirement Documentation

- ***Team Members Role:*** *Highlight the role of team members [e.g. Person A – Role Backend Lead – Fast API Microservices & Schema Design]*

No	Name	Role
1	Khor Mei Shi	Frontend Developer
2	Fong Ai Qi	Backend Developer
3	Tejeshwar	Database Developer
4	Oon Pei Ching	Database Developer
5	Haarristhiven Thamotharan	AI Prompt Engineer

-
- ***Recommendations:*** *If any recommendation is applicable in terms of scalability, performance and reliability [e.g. using Redis Cache for Menu data]*

-

	Recommendation	Reason
Scalability	Use Redis Cache for menu data	Repeated database queries for the same menu data slow down response time and increase database load. Redis stores data in memory for sub-millisecond access, reducing DB queries from thousands to just one.
Performance	Use CDN for static assets	CDN delivers files from the nearest server and cuts load times from seconds to milliseconds.
Reliability	Use AWS RDS read replicas for reporting	Provides failover target and keeps system available during heavy reports.

-

© UMHackathon 2026